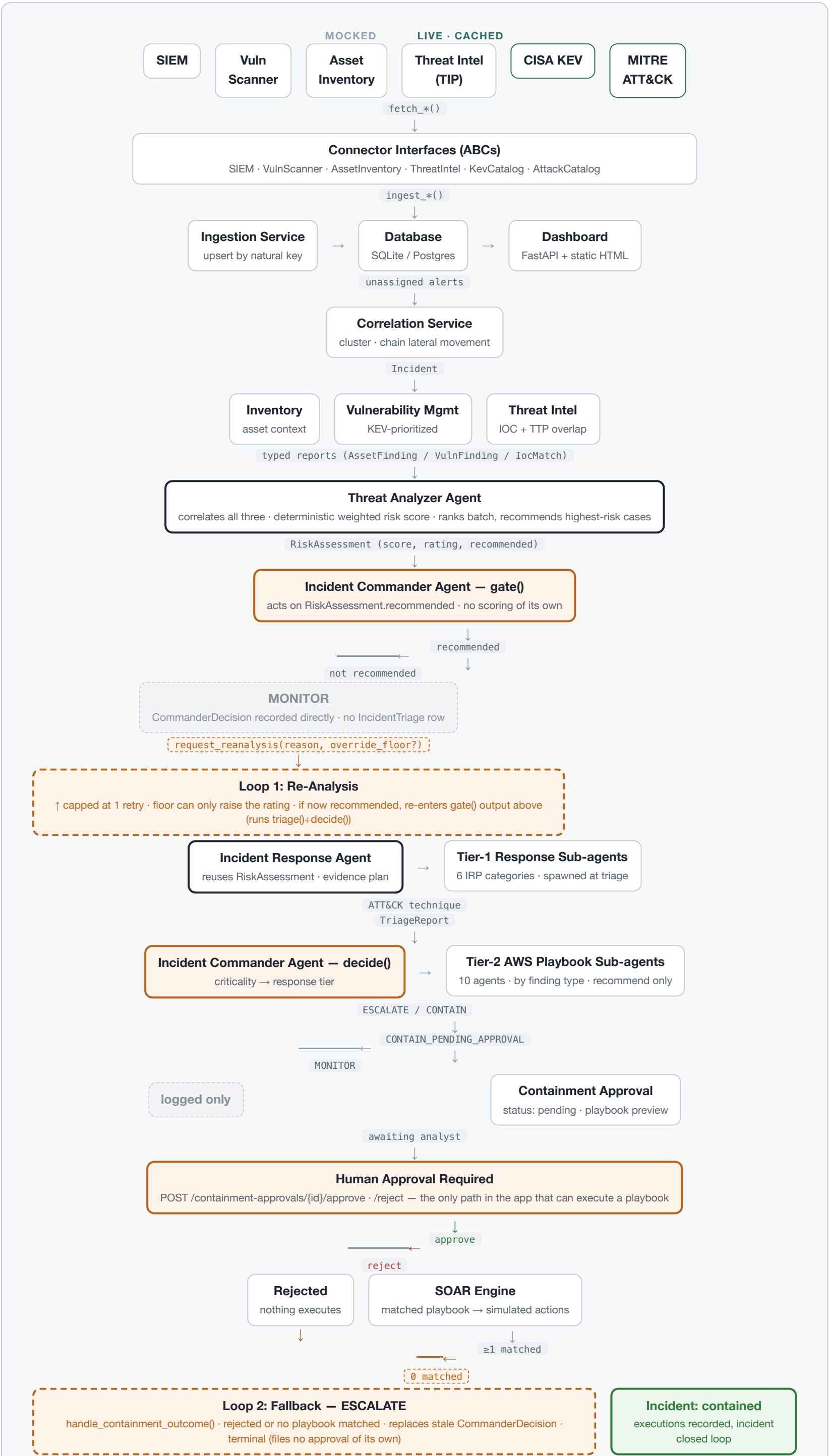


vbg_defense_orchestrator — pipeline map

How a raw feed becomes a triaged incident — a Threat Analyzer Agent correlates and risk-ranks every incident before the Incident Commander ever sees it, the Commander's gate acts on that recommendation, and its final response-tier call comes after triage. Two bounded feedback loops (orange, dashed) let a human send a decision back for reconsideration; the one gate no automated path is allowed to skip is still the human approval below.



The **Threat Analyzer Agent** now sits ahead of the Commander entirely: it correlates Inventory/Vulnerability/ Threat Intel data into one deterministic risk score per incident, ranks the whole batch by that score (`risk_rank`, 1 = highest), and recommends only high/critical-rated cases onward. The Commander's **gate()** just acts on that recommendation — no scoring logic of its own — and the Incident Response Agent reuses the same `RiskAssessment` instead of re-querying. Everything below **Human Approval Required** is still the one gate no automated path can skip: **reject** records the decision, or **approve** hands the incident to the SOAR Engine — the only code path in the app that can execute a SOAR playbook. Two dashed orange **feedback loops** let a human commander send a decision back rather than accept a dead end: **Loop 1** disputes a skipped incident's risk rating; **Loop 2** falls back to **ESCALATE** when containment was rejected or turned out to match no playbook. Both are bounded — neither can iterate more than once.

- ☐ hub agent (thick border)
- ☐ Incident Commander (gate / decide)
- ☐ terminal — nothing executes
- ☐ terminal — remediation applied
- ☐ feedback loop (bounded, human-initiated)

THE THREAT ANALYZER IS NEW

`ThreatAnalyzerAgent.analyze()` is the canonical scoring implementation — correlation confidence, severity, asset criticality/exposure, KEV-listed vulns, threat-intel overlap — and the only place that logic lives. It persists a `ThreatAnalysis` row and hands back a `RiskAssessment` that both the Commander's `gate()` and the IR Agent's `triage()` consume directly, with no re-querying.

TWO DISPATCH TIERS, TWO TRIGGERS

Tier-1 spawns off the IR Agent at triage, keyed by ATT&CK technique ID. Tier-2 spawns off the Commander's `decide()` call, keyed by GuardDuty/CloudTrail finding type. Both only ever recommend.

THE APPROVAL GATE IS ABSOLUTE

`approval_service.approve_containment()` is the sole caller of `soar_engine.evaluate_and_execute()`. There is no risk score, however high, that skips it.

FEEDBACK LOOPS ARE BOUNDED, NOT ARGUMENTS

`request_reanalysis()` allows exactly one retry (`ThreatAnalysis.revision`) and its floor can only raise a rating, never suppress one. `handle_containment_outcome()` always terminates at `ESCALATE`, which files no approval of its own — neither loop can run twice.